

Toy Model of Superposition

A toy model (Elhage et al., 2022b) investigates the hypothesis that neural networks can represent more *features* than the number of neurons by encoding real-world *concepts* in a compressed manner. The model considers a high-dimensional vector $\mathbf{x}$, where each element x_i corresponds to a feature capturing a real-world concept, represented as a random vector with varying importance determined by a weight a_i . These features are assumed to have the following properties:

- i.) **Concept sparsity:** Real-world concepts occur sparsely.
- ii.) **More concepts than neurons:** The number of potential concepts vastly exceeds the available neurons.
- iii.) **Varying concept importance:** Some concepts are more important than others for the task at hand.

The input vector $\mathbf{x}$ represents features capturing these concepts, defined by a sparsity level S and an importance level a_i for each feature x_i , reflecting the sparsity and varying importance of the underlying concepts. The model dynamics involve transforming $\mathbf{x}$ into a hidden representation $\mathbf{h}$ of lower dimension, and then reconstructing it as $\mathbf{x}'$:

$$\mathbf{h} = \mathbf{W}\mathbf{x}, \quad \mathbf{x}' = \text{ReLU}(\mathbf{W}^\top \mathbf{h} + \mathbf{b}).$$

The network’s performance is evaluated using a loss function $\mathcal{L}$ weighted by the feature importances a_i , reflecting the importance of the underlying concepts:

$$\mathcal{L} = \sum_{\mathbf{x}} \sum_i a_i (x_i - x'_i)^2.$$

This toy model highlights neural networks’ ability to encode numerous features representing real-world concepts into a compressed representation, providing insights into the superposition phenomenon observed in neural networks trained on real data.

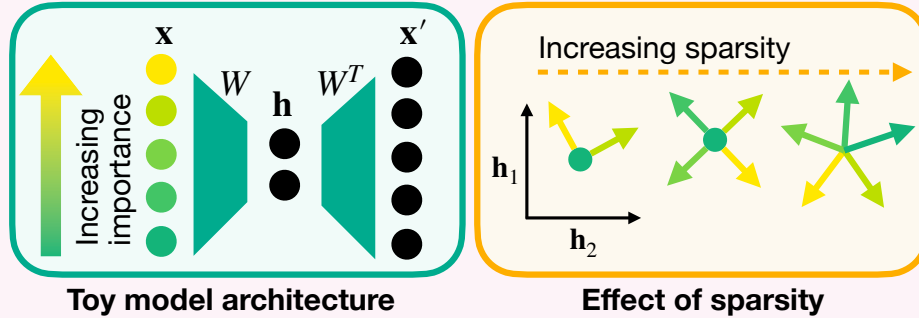


Figure 4: Illustration of the toy model architecture and the effects of sparsity. (left) Transformation of a five-feature input vector $\mathbf{x}$ into a two-dimensional hidden representation $\mathbf{h}$, and its reconstruction as $\mathbf{x}'$ using the weight matrix $\mathbf{W}$ and its transpose, with feature importance indicated by a color gradient from yellow to green. (right) The effect of increasing feature sparsity S on the encoding capacity of the network, highlighting the network’s enhanced ability to represent features in superposition as sparsity increases from 0 to 0.9, illustrated by arrows in the activation space $\mathbf{h}$, which correspond to the columns of the matrix $\mathbf{W}$.

be desirable as it allows models to represent more concepts with limited compute, making training cheaper. Overall, engineering monosemanticity has proven challenging (Bricken et al., 2023) and may be impractical until we have orders of magnitude more compute available.

Definition 2: Feature (Alternative)

Features are elements that a network would ideally assign to individual neurons if neuron count were not a limiting factor (Bricken et al., 2023). In other words, *features* correspond to the disentangled *concepts* that a larger, sparser network with sufficient capacity would learn to represent with individual neurons.

Another approach is to train networks in a standard way (creating polysemanticity) and use post-hoc analysis to find the feature directions in activation space, for example, with Sparse Autoencoders (SAEs). SAEs aim to find the true, disentangled features in an uncompressed representation by learning a sparse overcomplete basis that describes the activation space of the trained model (Bricken et al., 2023; Sharkey et al., 2022b; Cunningham et al., 2024) (also see Section 4.2).

If not neurons, what are features then? We want to identify the fundamental units of neural networks, which we call *features*. Initially, neurons seemed likely candidates. However, this view fell short, particularly in transformer models where neurons often represent multiple concepts, a phenomenon known as polysemanticity. The superposition hypothesis addresses this, proposing that due to limited representational capacity, neural networks compress numerous features into the confined space of neurons, complicating interpretation.

This raises the question: *How are features encoded if not in discrete neuron units?* While a priori features could be encoded in an arbitrarily complex, non-linear structure, a growing body of theoretical arguments and empirical evidence supports the hypothesis that features are commonly represented linearly, i.e., as linear combinations of neurons – hence, as directions in representation space. This perspective promises to enhance our comprehension of neural networks by providing a more interpretable and manipulable framework for their internal representations.

Hypothesis 2: Linear Representation

Features are directions in activation space, *i.e.*, linear combinations of neurons.

The *linear representation* hypothesis suggests that neural networks frequently represent high-level features as linear directions in activation space. This hypothesis can simplify the understanding and manipulation of neural network representations (Nanda et al., 2023b). The prevalence of linear layers in neural network architectures favors linear representations. Matrix multiplication in these layers most readily processes linear features, while more complex non-linear encodings would require multiple layers to decode.

However, recent work by Engels et al. (2024) provides evidence against a strict formulation of the linear representation hypothesis by identifying circular features representing days of the week and months of the year. These multi-dimensional, non-linear representations were shown to be used for solving modular arithmetic problems in days and months. Intervention experiments confirmed that these circular features are the fundamental unit of computation in these tasks, and the authors developed methods to decompose the hidden states, revealing the circular representations.

Establishing non-linearity can be challenging. For example, Li et al. (2023a) initially found that in a GPT model trained on Othello, the board state could only be decoded with a non-linear probe when represented in terms of white and black pieces, seemingly violating the linearity assumption. However, Nanda (2023c); Nanda et al. (2023b) later showed that a linear probe sufficed when the board state was decoded in terms of "one's own" and "the opponent's" pieces, reaffirming the linear representation hypothesis in this case. In contrast, the work by Engels et al. (2024) provides a clear and convincing existence proof for non-linear, multi-dimensional representations in language models.

While the linear representation hypothesis remains a useful simplification, it is important to recognize its limitations and the potential role of non-linear representations (Sharkey et al., 2022a). As neural networks continue to evolve, ongoing reevaluation of the hypothesis is crucial, particularly considering the possible

emergence of non-linear features under optimization pressure for interpretability (Hubinger, 2022). Alternative perspectives, such as the polytope lens proposed by Black et al. (2022), emphasize the impact of non-linear activation functions and discrete polytopes formed by piecewise linear activations as potential primitives of neural network representations.

Despite these exceptions, empirical evidence largely supports the linear representation hypothesis in many contexts, especially for feedforward networks with ReLU activations. Semantic vector calculus in word embeddings (Mikolov et al., 2013), successful linear probing (Alain & Bengio, 2016; Belinkov, 2021), sparse dictionary learning (Bricken et al., 2023; Cunningham et al., 2024; Deng et al., 2023), and linear decoding of concepts (O’Mahony et al., 2023), tasks (Hendel et al., 2023), functions (Todd et al., 2023), sentiment (Tigges et al., 2024), refusal (Arditi et al., 2024), and relations (Hernandez et al., 2023; Chanin et al., 2023) in large language models all point to the prevalence of linear representations. Moreover, linear addition techniques for model steering (Turner et al., 2023; Sakarvadia et al., 2023a; Li et al., 2023b) and *representation engineering* (Zou et al., 2023) highlight the practical implications of linear feature representations.

Building upon the linear representation hypothesis, recent work investigated the structural organization of these linear features within activation space. Park et al. (2024) reveal a geometric framework for categorical and hierarchical concepts in large language models. Their findings demonstrate that simple categorical concepts (e.g., mammal, bird) are represented as simplices in the activation space, while hierarchically related concepts are orthogonal. This geometric analysis aligns with earlier observations on feature clustering and splitting in neural networks (Elhage et al., 2022b). It suggests that the linear features are not merely scattered directions but are organized to reflect semantic relationships and hierarchies.

3.3 Circuits as Computational Primitives and Motifs as Universal Circuit Patterns

Having defined features as directions in activation space as the fundamental units of neural network representation, we now explore their computation. Neural networks can be conceptualized as computational graphs, within which *circuits* are sub-graphs consisting of linked features and the weights connecting them. Similar to how features are the representational primitive, circuits function as the computational primitive (Michaud et al., 2023) and the primary building block of these networks (Olah et al., 2020).

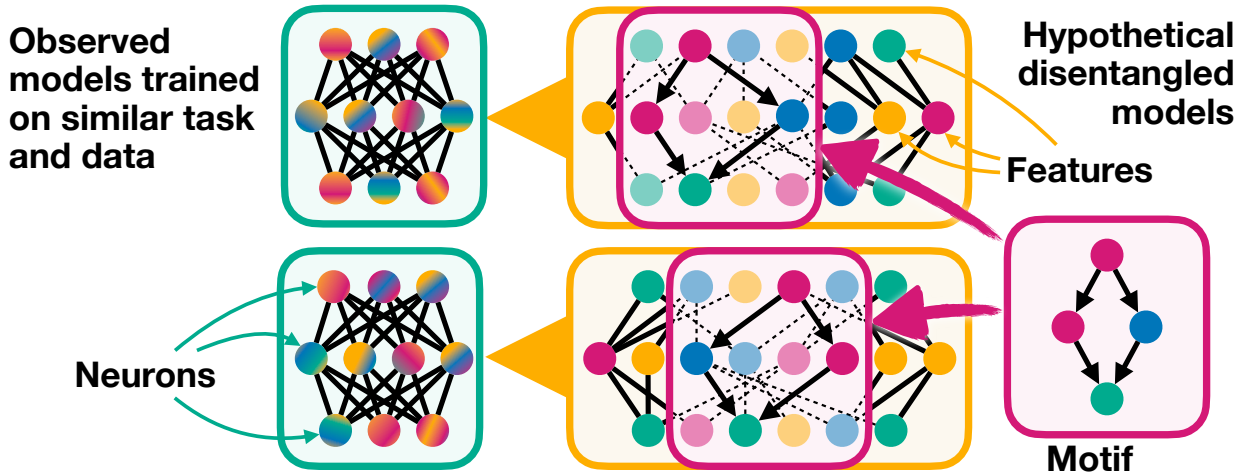


Figure 6: Comparing observed models (left) and corresponding hypothetical *disentangled* models (right) trained on similar tasks and data. The observed models show different neuronal activation patterns, while the dissection into feature-level *circuits* reveals a *motif* - a shared circuit pattern emerging across models, hinting at *universality* - models converging on similar solutions based on common underlying principles.

The decomposition of neural networks into circuits for interpretability has shown significant promise, particularly in small models trained for specific tasks such as addition, as seen in the work of Nanda et al. (2023a) and Quirke & Barez (2023). Scaling such a *comprehensive* circuit analysis to broader behaviors in large